SLM Code Architecture: which function calls which

From the moment `full_demo.py` is run to the text shown on each screen. Thick arrows = the order of the pipeline steps. Thin arrows = "this function calls that function". Box colour = the file the function lives in.

1 · START-UP (once, when the demo starts)

full_demo.py
main()
the program you run

then

full_demo.py
menu loop (while True)
waits for 1, 2, 3, w, s, r, q

slm_pipeline.py
SLMPipeline.__init__()
builds everything once

speech_input.py
is_available()
microphone + Vosk installed?

tokenizer.py
SimpleTokenizer(ALL_VOCAB)
word ↔ ID table, 95 IDs

slm_pipeline.py
_load("trained_picker.pt")
rebuilds the word picker

PyTorch
torch.load()
reads the saved numbers

word_picker.py
WordPicker(...) + eval()
75,411 parameters

slm_pipeline.py
_load("trained_tinyllama.pt")
rebuilds TinyLlama

decoder_tinyllama.py
TinyLlamaForGeneration(...)
133,376 parameters

slm_pipeline.py
ActiveContext()
empty conversation memory

Every later step uses what start-up built:
• self.tokenizer • self.picker (word picker)
• self.decoder (TinyLlama) • self.context (memory)

Shared files imported at the top of slm_pipeline.py:
vocab_pool.py → ALL_VOCAB, CONFIRMED_GESTURES, CATEGORIES (LOOKUP_TABLE, SIGNS, PEOPLE…)
onnx_config.py → GLOSS_MAP, normalize_input()
make_labels.py → normalize_word()
word_picker.py → WordPicker, restrict_to_signs()
decoder_tinyllama.py → TinyLlamaForGeneration

option 1 (signs from the camera / typed signs)

option 2 (typed) or 3 (microphone)

2 · ND2D path (hearing → Deaf)

calls each step below, top to bottom

runs the whole path
non_deaf_to_deaf()
returns a Turn

Voice Recording
voice_recording()
→ speech_input.py **record()** — 5 s from the microphone
→ sounddevice **sd.rec() + sd.wait()** — 16,000 samples/second

Speech to Text
speech_to_text()
→ speech_input.py **transcribe()** — audio → words
→ Vosk (pre-trained) **Model() + KaldiRecognizer()** — offline, 40 MB model

Normalization
normalization()
→ onnx_config.py **normalize_input()** — checks the input is text
→ slm_pipeline.py **known_form()** — trained form of a synonym
→ slm_pipeline.py **picker_training_words()** — reads labels_train.jsonl
→ onnx_config.py **GLOSS_MAP** — mom → mother, house → home

Reduction
reduction()
→ vocab_pool.py **ALL_VOCAB → LOOKUP_TABLE** — keeps the 91 known words

Tokenization
tokenize()
→ tokenizer.py **token_to_id[word]** — word → ID

Embedding
embedding(ids, "speech")
→ word_picker.py **WordPicker.embed()**
→ encoder_distilbert.py **DistilBertEmbeddings.forward()** — word 95×64 + position 16×64

Contextualize
contextualize()
→ word_picker.py **WordPicker.contextualize()**
→ encoder_distilbert.py **TransformerBlock.forward() ×2** — self-attention + FFN

Space Restriction
space_restriction()
→ word_picker.py **WordPicker.heads()** — tag 64→17 · question 64→2
→ word_picker.py **restrict_to_signs()** — only signs · negation → not
→ make_labels.py **normalize_word()** — word → sign name
→ SciPy **linear_sum_assignment()** — Hungarian: one slot each

Active Context
self.context …
→ slm_pipeline.py **ActiveContext.resolve_pronoun()** — he/she → remembered person
→ slm_pipeline.py **ActiveContext.note_hearing()** — remembers who / question
→ make_labels.py **normalize_word()**

Transcribed Word
transcribed_words()
→ slm_pipeline.py **" ".join(glosses)** — e.g. "where mother"

Display
display()
→ slm_pipeline.py **turn.output = text** — path returns the Turn to main()
→ full_demo.py · main() then calls **screen(reader, turn.output)** — prints the text box
→ Deaf person's screen **TEXT: where mother** — text facing the Deaf person

3 · D2ND path (Deaf → hearing)

calls each step below, top to bottom

runs the whole path
deaf_to_non_deaf()
returns a Turn

Translated Word
transcribed_signs()
→ onnx_config.py **normalize_input()** — text or list of words
→ tokenizer.py **_clean_and_split()** — lowercase, split
→ make_labels.py **normalize_word()** — mama → mother; keep 34 signs

Tokenization
tokenize()
→ tokenizer.py **token_to_id[word]** — sign → ID

Embedding
embedding(ids, "signs")
→ decoder_tinyllama.py **embed_source()** — sign table 95×64, no position

Active Context
self.context …
→ slm_pipeline.py **ActiveContext.fill_subject()** — adds a missing person
→ slm_pipeline.py **embedding() + torch.cat()** — person's vector joined

Sentence Completion
sentence_completion()
→ decoder_tinyllama.py **generate_from_memory()** — one word at a time
→ decoder_tinyllama.py **decode() → TinyLlamaBlock ×2** — self-attn, cross-attn, SwiGLU
→ tokenizer.py **decode()** — IDs → words
→ slm_pipeline.py **to_declarative()** — answer form
→ slm_pipeline.py **covered()** — safety net
→ onnx_config.py **GLOSS_MAP** — sign's other forms

Active Context
self.context.note_deaf()
→ make_labels.py **normalize_word()** — remembers the person

Display
display()
→ slm_pipeline.py **turn.output = text** — path returns the Turn to main()
→ full_demo.py · main() then calls **screen(reader, turn.output)** — prints the text box
→ Hearing person's screen **TEXT: mother is home** — text facing the hearing person

slm_pipeline.py · word_picker.py · encoder_distilbert.py · decoder_tinyllama.py · tokenizer.py · vocab_pool.py / onnx_config.py / make_labels.py · speech_input.py · full_demo.py (the live demo)
outside library (sounddevice, Vosk, SciPy, PyTorch) · screen output (text)
ND2D path: next step · D2ND path: next step · calls this function

How to read the architecture

The figure is the **algorithm of our SLM program**: what runs first, which function each step calls, and in which file that function is written. It does not include the sign recognizer (vision module); for the Deaf → hearing path, the recognized signs are typed in for now.

- **Top to bottom is the order in time.** Section 1 (start-up) runs once. Sections 2 and 3 run every time someone talks: the menu picks one of the two paths.
- **Orange box with a red or blue name** = one box of the adviser's pipeline. Each one is one method of the class `SLMPipeline` in `slm_pipeline.py` (the method name is under the box name).
- **Thick red arrow** = next step of hearing → Deaf. **Thick blue arrow** = next step of Deaf → hearing. The steps are called in this order by the runner method at the top of each lane (`non_deaf_to_deaf()` or `deaf_to_non_deaf()`).
- **Thin grey arrow** = "calls". The second column is what the step calls; the third column is what *that* function calls.
- **Dashed box** = a library we did not write (sounddevice, Vosk, SciPy, PyTorch).
- **White box with a thick border** = what appears on the screen. **Both outputs are text**: the Deaf person reads the sign words (e.g. "where mother"), the hearing person reads an English sentence (e.g. "mother is home").

One real conversation, traced through the code

Actual output of `slm_pipeline.py` with the trained models (two turns in a row).

Turn 1 · hearing person types "Where is your mom?" → `non_deaf_to_deaf()`		
1	Voice Recording	skipped: typed input (option 2)
2	Speech to text	typed: 'Where is your mom?'
3	Normalization	where is your mother [mom → mother]
4	Reduction	where is your mother [nothing removed]
5	Tokenize	[88, 41, 94, 50]
6	Embedding	4 words × 64 numbers
7	Contextualize	2 transformer blocks: each word now reflects the whole sentence
8	Space Restriction	where = slot 1, is = drop (no sign), your = drop, mother = slot 2 → where mother
9	Active Context	remembers mother
10	Transcribed Words	where mother
11	Display	**TEXT on the Deaf person's screen: where mother**

Turn 2 · Deaf person signs "home" → `deaf_to_non_deaf()`		
1	Transcribed Words	home
2	Tokenize	[36]
3	Embedding	1 sign × 64 numbers
4	Active Context	added 'mother' from earlier
5	Sentence Completion	mother is home [answer form: 'is mother home' → 'mother is home']
6	Display	**TEXT on the hearing person's screen: mother is home**

In turn 2, Active Context remembered "mother" from turn 1 and added it, because the Deaf person signed only "home".

The files and where they appear

File	What it holds	Steps that use it
`full_demo.py`	The program you run. `main()` builds the pipeline, shows the menu, calls one of the two paths, then prints the result with `screen()`.	start-up, menu, Display
`slm_pipeline.py`	The pipeline itself: class `SLMPipeline` with one method per box, the two runner methods, `ActiveContext`, `to_declarative()`, `covered()`.	every step
`speech_input.py`	Microphone recording and the pre-trained Vosk speech-to-text.	Voice Recording, Speech to text
`tokenizer.py`	Word ↔ ID table (95 IDs) and text cleaning.	start-up, Tokenize, Translated Word, Sentence Completion
`word_picker.py`	The hearing → Deaf model (75,411 parameters) and `restrict_to_signs()`.	Embedding, Contextualize, Space Restriction
`encoder_distilbert.py`	Building blocks the word picker is made of: the embedding tables and the transformer block.	inside Embedding and Contextualize
`decoder_tinyllama.py`	The Deaf → hearing model, TinyLlama (133,376 parameters).	Embedding (signs), Sentence Completion
`vocab_pool.py`	Word lists: the 91-word lookup table, the 34 signs, the people.	Reduction, Space Restriction
`onnx_config.py`	`GLOSS_MAP` (synonyms) and `normalize_input()`.	Normalization, Translated Word, safety net
`make_labels.py`	`normalize_word()`: turns a word into its sign name.	Space Restriction, Active Context, Translated Word
`trained_picker.pt` `trained_tinyllama.pt`	The saved numbers (weights) of the two trained models, read with `torch.load()` at start-up.	start-up

Not on the figure because they do not run during a conversation: `generate_dataset_v4.py` (makes the training data), `train_picker.py` and `train_tinyllama.py` (train the models and save the `.pt` files), `Conversation_test.py` (tests). They run *before*, offline.

What to say when presenting it

> "This is the architecture of our SLM code. When the program starts, `main()` in `full_demo.py` creates one `SLMPipeline`. Its `__init__` builds the tokenizer, loads our two trained models from their `.pt` files, and creates an empty Active Context memory.
>
> When the hearing person speaks, `main()` calls `non_deaf_to_deaf()`. That method calls the steps of the pipeline in order: Voice Recording and Speech to text use `speech_input.py`; Normalization and Reduction clean the words and keep only the words in our lookup table; Tokenize turns them into IDs; Embedding, Contextualize and Space Restriction run our word-picker model, which is built from the DistilBERT-style blocks; Active Context resolves he/she and remembers the person. The result is shown as **text** on the Deaf person's screen.
>
> When the Deaf person signs, `main()` calls `deaf_to_non_deaf()`. The signs are tokenized and embedded, Active Context adds a missing person from memory, and Sentence Completion runs TinyLlama, which writes the English sentence one word at a time. The sentence is shown as **text** on the hearing person's screen."